

Neuro-Formal Verification: Agentic Language-Agnostic Formal Program Reasoning

Shuvendu K. Lahiri
Microsoft Research

Abstract

Formal verification provides the strongest correctness guarantees available for software, and verification-aware languages can produce sound, machine-checked proofs. Recent AI coding agents have dramatically lowered the cost of constructing such proofs. Yet these benefits reach few mainstream developers: most use languages without formal-verification support, and formalizing properties and modeling execution environments require expertise in formal methods. Proof therefore remains reserved for a small number of notable artifacts that are used in practice, while production software is attested primarily through review and testing.

We introduce *neuro-formal verification* (NFV), which brings this automation to mainstream programming languages. An AI coding agent formalizes a source-level verification problem into a proof obligation in a verification-aware language, which is discharged by an established sound verifier aided by agentic proof search. NFV uses staged, goal-blind transformations to reduce the risk of proving an artifact that does not faithfully represent the source program, property, or environment. Since NFV cannot ensure the soundness of the formalization — a correspondence that can only be loosely defined in the presence of informally stated environment assumptions — it optimizes for empirical accuracy rather than end-to-end soundness, while insisting on machine-checked evidence for every verdict.

Experiments with current frontier models on a balanced dataset of correct and buggy solutions to Python programming problems demonstrate the effectiveness of our approach. NFV with DAFNY correctly resolves 57% of all entries, with 92% precision among its verdicts; with a CBMC backend, it produces a counterexample for 63% of the buggy programs at 90% precision. In contrast, an *LLM-as-judge* baseline achieves only 72% precision despite answering every entry without producing any checkable artifact; an unstaged agent-verifier combination proves 98% of both the correct and the known-buggy programs, yielding only 50% precision. Together, they confirm that both proofs and staging benefit an AI agent’s formal program reasoning.

1 Introduction

Formal program verification refers to the use of mathematical logic to derive a mathematical *proof* of a formal property ϕ against a formal model of a program P operating in an execution environment E . The idea dates to Turing [1] and Hoare [2], and it has been held out ever since as the strongest available form of assurance for program correctness. Decades of work have made it real. A range of mature verification-aware languages and proof assistants — DAFNY [45], F* [9], Verus [15], LEAN [3], ROCQ [7], ISABELLE/HOL [8] — combine SMT-backed [5, 6] automation with user-supplied proof structure in varying proportions. They have been used to prove substantial systems correct, including some that are widely used [10, 11, 12, 13]. Such success stories are, however, relatively few and often involve experts in those languages and proof assistants. They are typically green-field projects in which code and proof are co-developed under a methodology that takes years to establish.

A proof of ϕ for a program P in a mainstream language L faces two additional obstacles. First, L ’s constructs must be formalized — in a program logic or an intermediate verification language (IVL) such as BOOGIE [16]. Efforts in that direction, from VCC [14] to Verus [15], are real but slow and cover a handful of languages. Even for a language as studied as C there is no single verifier for the whole language, as CBMC [39], VCC and Frama-C [38] embody different memory models, specification formalisms and proof systems, each catering to a different appetite for automation. Second, a proof about P is a proof about P running in an environment E — the precondition the developer has in mind, the semantics of the libraries P calls, the external state it reads and updates — all of which have to be inferred from documentation and world knowledge. The SLAM/SDV developers, for instance, spent decades building and maintaining accurate models of the operating system in order to verify device drivers against it [41]. As a result, for the code that actually ships, developers have stuck with code review and testing as the cost-effective way to obtain assurance.

Two developments of the past year change the calculation. First, developers increasingly produce code with AI coding agents, a practice sometimes called “vibe coding” [37], which raises the demand for assurance. Second, those same agents turn out to be unexpectedly good at writing code *and proofs* in exactly the verification-aware languages whose expert supply has long been a bottleneck [42, 43, 4].

This motivates the question this paper sets out to answer:

Can we harness the proof automation of mature verification-aware languages for mainstream developers in a language-agnostic way?

Our central conjecture is that a technique capable of distinguishing *correct programs* (those that satisfy a stated property in a given environment) from *incorrect programs* (those that do not), with extremely high empirical accuracy — say, 99.99% — while producing a checkable artifact that explains each verdict could be invaluable for auditing code, understanding it, and calibrating trust in it, especially when the code is AI-generated.

In this paper, we describe *neuro-formal verification* (NFV) as one possible way to realize this vision. An effective neuro-formal verifier would allow a developer using a mainstream programming language L to pose *formal reasoning questions* (say, as formal assertions in L) and obtain answers that are (a) empirically accurate without necessarily providing end-to-end guarantees of soundness or completeness, (b) produced at the push of a button by default, and (c) accompanied by a proof over formal models of the program, the property, and the environment.

1.1 Neuro-formal verification: the whole picture

The inputs. A developer has a program P_L in some source language L ; a property S_{nl} they care about, written in whatever notation they already use — a docstring, an issue, a comment; and an environment E in which the program executes — including the context that calls it and the libraries and services it calls — described by documentation E_{nl} .

The workflow. Figure 1 lays out five stages that turn those inputs into a checked proof over a model of a program and return it in the developer’s own terms. The workflow is parameterized by a verification-aware language F with first-class formal verification support.

1. **Intent formalization** [49]. Synthesize from E_{nl} a formal specification E_F in F — the contracts of the libraries the program calls and any preconditions stated by the documentation — and from S_{nl} an assertion S_L in the source language that expresses what the developer

meant. Both are claims about intent, and both should be *attested* against every artifact available — the documentation, the tests, and the source when available.

2. **Program translation**, $P_L \rightarrow P_F$, into the formal language F of some mature verifier. This step can range from purely symbolic (a compiler, where a faithful front end exists) through mixed approaches to purely neural (this paper).
3. **Specification translation**, $S_L \rightarrow S_F$, along the same spectrum.
4. **Proof search over F** , discharged by the verifier for F and driven agentially: an agent proposes invariants, lemmas, and tactics; the verifier checks each proposal, so the search may use neural methods freely without putting the verifier’s soundness at risk.
5. **Proof mapping**. Map a successful formal proof $Proof(P_F, S_F, E_F)$ into a *proof sketch* stated over P_L , using the developer’s own variables and naming the assumptions in terms of E_{nl} .

The workflow should not be construed as end-to-end formal verification that P_L satisfies S_L . The source-language semantics, the environment description, and the translations are not all formally attested. We therefore use the term *formal program reasoning*.

Proof beyond the final verdict. Formal proof is not confined to establishing S_F for P_F under E_F . Intent formalization can also produce a test suite T_F , either extracted from library artifacts or generated by agents, together with proofs of the soundness, completeness, and preciseness of E_F with respect to T_F [49]. Similarly, the path from S_{nl} to S_F can produce tests and proofs attesting S_F . These artifacts strengthen the attestation of the formalized environment and specification, even though they do not establish end-to-end correctness. More generally, the explicit formal artifacts P_F , S_F , and E_F give users inspectable accounts of the program model, the property being checked, and the assumptions under which the result holds. A user can review these artifacts, identify mismodeling, and provide feedback that improves subsequent formalization. Users can also vet familiar concrete tests rather than inspect S_F and E_F directly, reducing the formal-methods expertise required of them.

1.2 An example

Let us illustrate the workflow for this paper with a simple two-line Python program. We restrict the list elements to be exact rationals (Python’s `fractions.Fraction`), so that the division operator `/` performs no rounding — an assumption needed for the specification to hold.

```
def shifted_mean(l):
    "Average the list after adding 1 to each element; l must be non-empty
    ."
    return sum(x + 1 for x in l) / len(l)

# postcondition
assert shifted_mean(l) == sum(l) / len(l) + 1
```

Informally, the correctness of this specification relies on

$$\text{len}(l) \neq 0$$

and

$$\text{sum}(\{x + 1 \mid x \in l\}) = \text{sum}(l) + \text{len}(l).$$

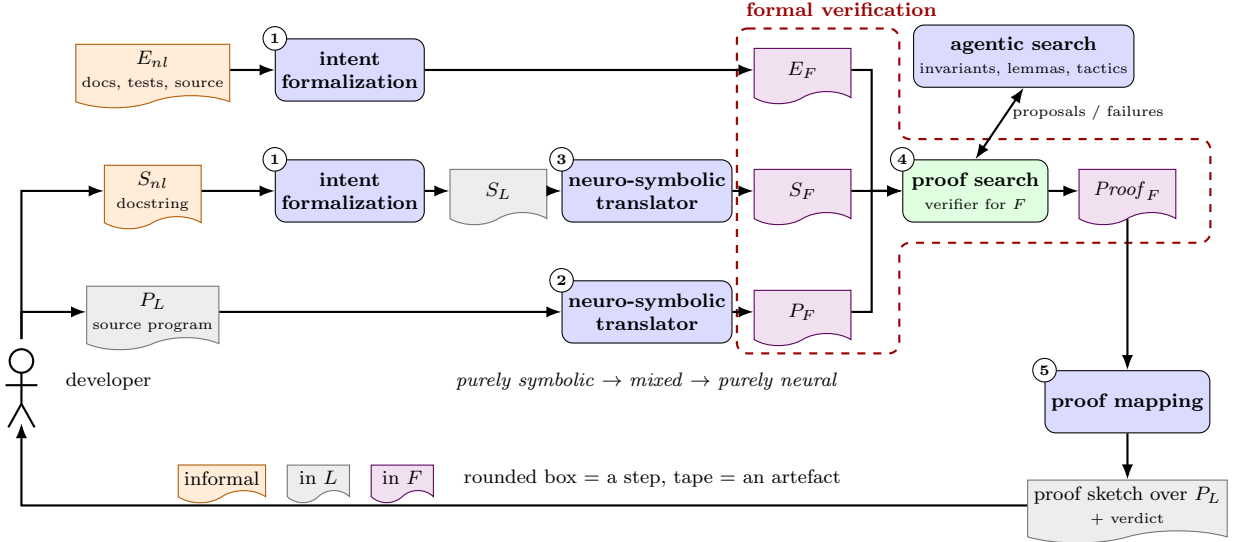


Figure 1: Neuro-formal verification. Circled numbers are the five stages enumerated above. Rounded boxes transform; tape-shaped nodes are the artifacts they consume and produce, colored by the language the artifact lives in — orange for informal English, gray for the source language L , purple for the formal language F . Blue is a step an LLM performs, green a step a verifier performs. The developer authors S_{nl} and P_L and reads the proof sketch that comes back. The dashed red region is the classical formal verification problem — given P_F , S_F and E_F , find a proof.

Stage 5 would not only return “verified” but also the proof-sketch reasoning, in natural language: *the specification holds because adding 1 to each of $\text{len}(l)$ elements adds $\text{len}(l)$ to their sum, provided the list is non-empty* — an artifact the developer can check.

The same example, worked in Dafny. Instantiating F with DAFNY makes the workflow concrete. Stage 1 starts from E_{nl} , which for this program is the documentation of Python’s `sum`¹ — all that is actually on offer about the library:

Return the sum of a ‘start’ value (default: 0) plus an iterable of numbers.
When the iterable is empty, return the start value.

Stage 1 formalizes this documentation as E_F , which in this example consists only of a recursive definition of `Sum`:

```
function Sum(l: seq<real>): real {
  if |l| == 0 then 0.0 else l[0] + Sum(l[1..]) }
```

`Sum` is attestable against the docstring: it is a recursive definition whose base and recursive cases can be read directly against the documented behavior.

Stages 2 and 3 then render P_F and S_F . Translating the generator expression `x + 1 for x in l` introduces `Shift` as part of the model of P_F ; it is not part of E_F :

¹The built-in’s runtime docstring — `sum.__doc__` in CPython 3.14 — quoted as it stands, less a closing sentence restricting it to numeric values.

```

function Shift(l: seq<real>): seq<real> {
  seq(|l|, i requires 0 <= i < |l| => l[i] + 1.0) }

method ShiftedMean(l: seq<real>) returns (r: real)
  requires |l| > 0
  ensures r == Sum(l) / (|l| as real) + 1.0
{ ShiftedSum(l);
  var n := |l| as real;
  r := Sum(Shift(l)) / n;
  DivDistrib(Sum(l), n); }

```

The two calls in the method are proof steps, not part of P_F . Stage 4 introduces both lemmas. **ShiftedSum** proves by induction that translating the generator preserves the expected relationship between the two sums. **DivDistrib** supplies the nonlinear arithmetic step that the verifier does not close automatically when n is symbolic:

```

lemma ShiftedSum(l: seq<real>)
  ensures Sum(Shift(l)) == Sum(l) + |l| as real
{ if |l| > 0 {
  assert Shift(l)[1..] == Shift(l[1..]);
  ShiftedSum(l[1..]); } }

lemma DivDistrib(a: real, n: real)
  requires n != 0.0
  ensures (a + n) / n == a / n + 1.0
{ assert n / n == 1.0;
  assert (a + n) / n == a / n + n / n; }

```

DAFNY checks both lemmas and, with their calls in **ShiftedMean**, discharges all seven obligations of the resulting file. Stage 5 would map this proof back to the proof-sketch reasoning stated above: *the specification holds because adding 1 to each of $\text{len}(l)$ elements adds $\text{len}(l)$ to their sum, provided the list is non-empty*. The current instantiation evaluated in this paper does not include this mapping.

The same example with a bug, refuted. Suppose the developer had written the reduction as a single addition on the total rather than one per element — $r := (\text{Sum}(l) + 1.0) / n$ in place of $r := \text{Sum}(\text{Shift}(l)) / n$, a one-token slip.

```

def shifted_mean(l):
    "Average the list after adding 1 to each element; l must be non-empty
    ."
    return (sum(l) + 1) / len(l) # bug

```

Now P_F does not meet S_F , and stage 5 should say so. The tempting move is to report the verifier's failure: DAFNY cannot prove the postcondition. However, a proof failure cannot be interpreted as a bug — an unproved goal is evidence about the proof search, not about the program.

What makes a refutation attestable is the same thing that makes a verification attestable — a proof that S_F fails. The program is left untouched, S_F is negated, and the witness is added as a precondition, so the verifier is asked to confirm a positive claim rather than to fail at a negative one:

```

method ShiftedMean(l: seq<real>) returns (r: real)
  requires |l| > 0
  requires l == [0.0, 0.0] // the witness
  ensures !(r == Sum(l) / (|l| as real) + 1.0) // S_F, negated
{
  var n := |l| as real;
  r := (Sum(l) + 1.0) / n; // the bug
  assert Sum(l) == l[0] + Sum(l[1..]);
  assert Sum(l[1..]) == l[1] + Sum(l[1..][1..]);
  assert l[1..][1..] == []; }

```

The witness $l = [0, 0]$ makes the buggy program return $1/2$ where S_F demands 1; the three asserts are the proof-repair loop unfolding `Sum` on that input, and DAFNY discharges the negated goal. On the Python list `[Fraction(0), Fraction(0)]` the buggy program itself returns `Fraction(1, 2)` where the specification demands `Fraction(1)`, and the `assert` raises `AssertionError`.

1.3 Contributions of this paper

This paper provides the first empirical instantiation of the NFV framework described in Figure 1. Specifically, we contribute:

- A dataset of 206 Python (program, specification) pairs — 103 correct references and 103 buggy mutants of them — derived from prior work [48].
- The NFV framework, including its goal-blind staged discipline and the gates and checks that enforce it.
- Two instances of the framework: a proof-based pipeline using DAFNY and a bounded-verification pipeline using CBMC.
- An empirical comparison against an *LLM-as-judge* baseline and an unstaged *agent-verifier* baseline, isolating the contributions of proof and of staging.

Scope of this instantiation. We begin at S_L , because the benchmark supplies an assertion synthesized from a docstring by prior work [48]. We synthesize E_F but do *not* formally attest it against tests describing E_{nl} , and both translation stages sit at the neural end of the spectrum. Finally, we stop at a verdict backed by a machine-checked proof in F rather than map that proof into a sketch over P_L ; evaluating such sketches requires a careful user study.

Within this scope, we demonstrate encouraging and timely results:

- With a flagship frontier model,
 - the DAFNY backend proves **70%** of the correct programs and refutes **46%** of the mutants at 94% and 89% precision, signifying the viability of the approach.
 - the combined pipeline issues a machine-checked verdict on 128 of the 206 entries, correctly resolving **57%** of all entries at **92%** precision.
 - CBMC refutes **63%** of the mutants at **90%** precision — the highest recall and precision for finding bugs in this dataset.
 - On this dataset, a majority of incorrect verdicts stem from incorrect precondition synthesis in E_F that can be improved with better intent-formalization methods.
- Less capable frontier models perform significantly worse on both verification and refutation — this capability may not have existed even a year ago.
- NFV outperforms the strawman *LLM-as-judge* baseline on this dataset: the judge answers every entry at 72% precision (compared to the 92% precision NFV provides through its ability to abstain when it cannot back its verdict with a proof artifact).

- NFV also outperforms an unstaged *agent-verifier* baseline, which proves 98% of the correct programs and 98% of the buggy ones at 50% precision, confirming the benefit of the staged discipline.

A dual reading. We have described the architecture from the perspective of a program verifier: a formal method that uses an LLM as its front end. It reads equally well from the other side, as *equipping an AI agent with a formal verifier*: the LLM is the agent, the pipeline is a tool it calls to discharge a claim about program correctness.

Organization. Section 2 describes the dataset, Section 3 the framework, Sections 4 and 4.2.1 the experiments and the errors that remain, Section 5 the related work, and Section 6 concludes with directions for future work.²

2 Dataset

This section describes a dataset we curate to study neuro-formal verification empirically.

2.1 Curation

Source. We select the programs and specifications from the NL2POSTCOND benchmark [48].³ This dataset builds on HumanEval [46], whose tasks provide (i) a natural-language *docstring* stating the intended behavior of the function, together with a handful of worked input/output examples, and (ii) a *reference* Python implementation P , taken to be correct with respect to a set of tests T augmented by EVALPLUS [47]. It augments each task with (iii) a set of postconditions synthesized from the task docstring and (iv) a set of mutants M that fail the tests. Each postcondition is labeled *sound* if it passes each test in T over P ; its *completeness* is the fraction of M killed by running the test inputs in T with the postcondition asserted. We take the postconditions from that repository’s `eval_results/gpt4_simple_no_ref.zip` — the GPT-4 SIMPLEPROMPTNOREF arm, whose `summary_eval_results.jsonl` carries the soundness and completeness verdicts and whose `individual_postcondition_results/` directory carries one file per candidate — and the mutants together with their killing inputs from `eval_results/distinct_code_mutants.jsonl.zip`.

Initial pool. For each task, we choose the specification that has the highest completeness score. Next, we choose the first mutant that the specification is confirmed to kill on an input in T . This provides us with 155 reference implementations (labeled correct) and 155 mutant implementations (labeled incorrect), where each pair shares a specification S .

Filtering. We then apply two filters. The first checks whether the reference fails the specification — the goal is to perform additional testing to remove any example where the specification is incorrect, assuming the reference implementation is correct. `claude-sonnet-4.5` was prompted for 100 diverse inputs per task and these were executed against P and S over all 155 references. We dropped 6 tasks where P was observed to violate S , and 33 more whose tests did not run to completion, leaving 116. Our approach is conservative — a falsifying input may itself violate a precondition the docstring states only in prose. The second filter drops 13 tasks whose postcondition constrains only the return *type*, an obligation that a typed target language F encodes in the method signature and that therefore does not exercise the program verifier. What remains is **206**

²The dataset, the pipeline and the scripts that produce every table and number reported here will be released shortly as open source.

³<https://github.com/microsoft/intent-formalization/tree/main/nl2postcondition-fse2024>

Table 1: nl2post-safe by library tier.

Tier	What the program touches	Tasks	Entries	Share
L0	nothing to axiomatize	21	42	20%
L1	total built-ins only	22	44	21%
L2a	collections, first-order only	25	50	24%
L2b	a function passed as an argument	26	52	25%
L3	module imports, unmodeled built-ins	9	18	9%
All		103	206	100%

Table 2: Size and control flow of the 206 programs. LOC counts distinct physical lines of the function under test carrying a syntax node; comments, blank lines, imports and the docstring are excluded. Paired by task, mutants are longer than their reference on 73 of 103 tasks and shorter on 18, a median gap of 2 lines.

	min	p25	median	mean	p75	p90	max
LOC	2	4	7	7.6	10	12	29

(a) Program size.

control flow	#	%	control flow	#	%
loop-free	92	45%	≥ 1 branch	144	70%
≥ 1 loop	114	55%	≥ 1 comprehension	37	18%
≥ 2 loops	24	12%			

(b) Control flow.

entries over 103 tasks, balanced by construction. Appendix A.1 lists every one of them with the postcondition it is scored against and the upstream identifiers that pin it, so the corpus can be reconstructed without rerunning the filtering steps that are not deterministic.

2.2 Characteristics

Table 1 and Table 2(a)–(b) characterize the resulting corpus along two axes: the library surface each program uses, and its size and control flow.

Tiers. Table 1 splits the corpus by **library tier** L0–L3, a function of the calls the Python source makes: a proxy for how much the LLM must formalize the environment, not for algorithmic difficulty. **L0** needs no axiom at all — arithmetic, indexing, control flow, `len`, `range`; **L1** adds total built-ins (`abs`, `sum`, `ord`); **L2a** is first-order collection use; **L2b** passes a function as an argument, so the specification must quantify over a body it cannot see; **L3** is a module import or any call outside these. Appendix B shows an example of each.

Size. Table 2(a) gives program size in terms of lines of code (LOC). The programs are small, in part because Python’s rich built-in and library functionality allows substantial behavior to be expressed in few lines. Table 2(b) summarizes control flow: a majority contain a loop and most a branch. The mutants are independently written solutions rather than minimal edits and are often longer than their references.

3 Neuro-formal verification (NFV)

This section instantiates the neuro-formal verification framework (Figure 1) by elaborating some of the components. First, we describe some syntactic *safeguards* to ensure that the AI agent does not simplify or alter the verification task midway through the pipeline (e.g., changing the translated source P_F during neural proof search); these are collected in Section 3.1.1. Second, we describe how to adapt the framework to perform a proof of refutation of the specification S_F (Section 3.1.2). Finally, we also extend the framework to accommodate bounded-program verifiers (such as CBMC or CORRAL) that search for witnesses over bounded unrolling of the program loops and recursion (Section 3.2).

3.1 The proof-based pipeline

We outline both the verification and refutation arms of this pipeline. Both directions roughly run the same sequence of steps and hand the result to a deductive verifier; they differ only in the goal. The steps refine the stages of Figure 1: the library theory (stage 1a) and the precondition (stage 1b) are the two halves of E_F produced by stage 1; transliterating the body is stage 2; rendering the specification is stage 3; and proof search and repair are stage 4. We instantiate the framework with a specific choice of L (Python) and F (DAFNY), but the concepts should be broadly applicable to other programming languages (C#, Java, JavaScript) and verifier backends (F*, LEAN). We choose DAFNY as our target verification-aware language for this work since it supports many features of modern imperative programming languages such as sequences and so lets the translation stay close to the source.

3.1.1 Verification proof

The verification arm aims to produce a proof of S_L over a translated program.

Stage 1a formalizes the Python library surface the program touches and stage 1b synthesizes preconditions from the docstring; both freeze before the specification is visible, so neither the library theory nor the precondition can be chosen to suit the goal.

Provenance tags. Stage 2 tags every emitted line with either `src:`, quoting its Python line verbatim, or `synth:`, marking it as having no source counterpart (declarations, invariants, `decreases` clauses). Untagged code is rejected — without that rule, rewritten logic hides in lines the audit cannot see. Translation has its own repair loop, but the only feedback is to run a syntax checker (e.g., `dafny resolve` in DAFNY): type and resolution errors. The loop sees no verifier goal, so it has no information with which to make the program *correct*, only enough to make it well-formed.

Figure 2 shows the partial artifact for our running example. Every line carries a provenance tag, and the two `src:` lines quote the Python verbatim. `Sum` is the only stage-1a content here: what Python’s `sum` means, written before the goal is visible. *Frozen* marks what proof search may not change — every `src:` line, the stage-1a theory, the stage-1b precondition and the goal itself — each re-checked mechanically on every repair round; the lemmas and the two proof calls at the bottom are what the repair loop contributed. `Shift` encodes the generator expression and so belongs to the translation that calls it. `ShiftedSum` and `DivDistrib` are the output of proof search — the facts the goal reduces to, found from the verifier’s failures — and both are lemmas *proved* in the target rather than assumed. The full artifact verifies at seven obligations.

In addition to the above checks to ensure the translated code and environmental specifications are not altered to achieve a proof, we also ensure that the precondition is not inconsistent and the

Python + specification

```
def shifted_mean(l):
    "Average the list after
    adding 1 to each
    element; l must be
    non-empty."
    return sum(x + 1
               for x in l) / len(l)

assert return_value ==
       sum(l)/len(l) + 1
```

Translated Dafny artifact

```
// synth: frozen library theory
// doc: sum(iterable, start=0) returns start plus the
// items of the iterable; on an empty iterable, start.
function Sum(l: seq<real>): real
{ if |l| == 0 then 0.0 else l[0] + Sum(l[1..]) }

// synth: translation helper, for the generator expression
function Shift(l: seq<real>): seq<real>
{ seq(|l|, i requires 0 <= i < |l| => l[i] + 1.0) }

// synth: proof, proved not assumed
lemma ShiftedSum(l: seq<real>)
  ensures Sum(Shift(l)) == Sum(l) + |l| as real
{ /* induction on l */ }
lemma DivDistrib(a: real, n: real) requires n != 0.0
  ensures (a + n) / n == a / n + 1.0
{ /* two asserts, both checked */ }

// src: source.py:1 | def shifted_mean(l):
method shifted_mean(l: seq<real>) returns (return_value: real)
  // synth: frozen precondition, from "non-empty"
  requires |l| > 0
  // synth: frozen postcondition
  ensures return_value == Sum(l) / (|l| as real) + 1.0
{
  // src: source.py:3 | return sum(x + 1 for x in l) / len(l)
  return_value := Sum(Shift(l)) / (|l| as real);
  // synth: proof
  ShiftedSum(l); DivDistrib(Sum(l), |l| as real);
}
```

Figure 2: The running example of Section 1.2 as the pipeline emits it. The bodies of `ShiftedSum` and `DivDistrib` are elided for space; both are proved, and Section 1.2 gives those proofs.

translation does not introduce unproven assumptions.

3.1.2 Refutation proof

Refutation asks the opposite question — is there an input class on which the specification *provably* fails — and reuses stages 1a, 1b and 2 byte-identically. The goal is negated *mechanically*, the postcondition S_F becomes $\neg S_F$. Next, the LLM proposes a violating input looking at only the Python source; that guess is installed as an additional precondition (this is an exception to the verification arm, where preconditions cannot be strengthened); and proof repair proves the negated goal with the body still frozen. Similar to the verification proof, this may require the synthesis of invariants, lemmas and intermediate assertions.

We also check a few additional facts specific to refutation. REACHABILITY requires that replacing the postcondition with *false* fails verification. GOAL SHAPE requires that only the inversion of S_F is present as a postcondition.

3.2 The bounded-verification-based pipeline

The second instantiation drops the proof and searches for the witness directly. It shares most of the stages, except that they address only refutation. A bounded model checker (such as CBMC, which operates over C) answers the refutation question natively: it searches for an execution violating

the goal assertion within a fixed unwinding, so a violating trace is a refutation and *every* other outcome, including a clean run, is an abstention. This arm also has no counterpart to agentic proof search or proof repair.

3.3 Proof-based versus bounded verification for refutation

Both pipelines can refute a program: the proof-based one by proving that a witnessing precondition forces a violation, the bounded one by finding a trace that exhibits it. Neither is complete, and the challenges each faces are different.⁴

Proof scales; search enumerates. A refutation (proof) search can scale to large modules due to its modular nature when paired with an effective (agentic) proof search automation. On the other hand, bounded verification is inherently non-scalable and is known to deteriorate with the depth of call chains. Ideas such as stratified inlining [55], that keeps callees abstract until a candidate trace demands them, may offer some relief — but cannot help with indefinite scaling.

Proof is easier to inspect. A proof is written in the vocabulary of the translated program, so the provenance tags map it back to the source line by line (Figure 2). A bounded counterexample is a trace, and in our experience an interprocedural trace spanning several tens of methods is hard to act on unless a concrete test can be generated from it.

Proof is incomplete under internal non-determinism. The witness is installed as a precondition, so the obligation is that *every* execution it admits violates the specification. A bug that manifests on some choice of internal non-determinism — a hash order, a clock, a service reply — cannot be discharged. Approaches based on *Incorrectness logic* [56] can alleviate this issue and also avoid unrolling loops with underapproximate loop summaries.

Bounded search failures can be spurious. For bounded verifiers, it is difficult to distinguish true violations from the ones admitted due to the environment E_F being under-constrained. On the other hand, a proof of refutation (whenever present) is a guarantee of a bug under the stated environment assumptions.

4 Evaluation

4.1 Setup

In this section we evaluate the efficacy of the NFV pipeline on our dataset. Unless stated otherwise the front end is GPT-5.6-SOL, OpenAI’s flagship model at the time of writing, and the target is DAFNY. Two budget parameters govern the proof search: an *attempt* discards the current translation and restarts from stage 1, while a *repair round* attempts to repair the proof failure through stage 4 only. All results use **5 attempts** and **10 repair rounds**, unless otherwise stated.

We measure *precision* as the number of correct verdicts over verdicts issued, and *recall* as the number of correct verdicts issued over the entries of that class. Abstention is not counted as a verdict. $F1$ is their harmonic mean, which we report because the two are traded against each other: abstaining more raises precision and lowers recall.

⁴We assume throughout that we cannot simply run P instead. Executing P may be unavailable or not cost-effective — P may be a component of a larger system (say a device driver) needing hardware, a build or a fixture the auditing tool does not have — or ϕ may speak of *ghost state* with no runtime representation. Combining NFV with testing opens up other interesting optimization possibilities not covered in this paper.

Table 3: DAFNY, the reference backend, on each task and its own population. Verification and refutation run at 5 attempts $\times$ 10 proof rounds. The pipeline row is composed from the two arms above — refutation first, verification on what it leaves open — so every row is the same backend. “wrong” counts verdicts contradicting the label.

task	population	issued	wrong	prec. (%)	rec. (%)	F_1
verification ($P \models S$)	103	77	5	93.5	69.9	0.80
refutation ($P \not\models S$)	103	53	6	88.7	45.6	0.60
pipeline (refute, then verify)	206	128	10	92.2	57.3	0.71

Table 4: The two tasks by library tier, one run each.

Tier	verification			refutation		
	proved	<i>bad</i>	<i>prec</i>	refuted	<i>bad</i>	<i>prec</i>
L0	14/21 (67%)	0	100%	10/21 (48%)	1	91%
L1	18/22 (82%)	2	90%	11/22 (50%)	0	100%
L2a	15/25 (60%)	1	94%	12/25 (48%)	2	86%
L2b	22/26 (85%)	0	100%	13/26 (50%)	1	93%
L3	3/9 (33%)	2	60%	1/9 (11%)	2	33%
All	72/103 (70%)	5	94%	47/103 (46%)	6	89%

4.2 The proof-based arm: Dafny

Table 3 gives the result on each task using the DAFNY verifier backend. Of the 206 (program, specification) pairs, comprising the correct references and the incorrect mutants, the verification arm (row marked “verification”) generates a correctness proof in DAFNY for 72 of the 103 correct reference implementations and issues proofs for 5 of the mutants labeled incorrect. This corresponds to a $\approx 70\%$ recall and $\approx 93.5\%$ precision. On the other hand, the refutation workflow (row marked “refutation”) produces a proof of violation for 47 of the 103 mutants while proving 6 reference implementations incorrect on some input. This corresponds to a $\approx 46\%$ recall and $\approx 89\%$ precision. In addition, we create another configuration (“pipeline”) where we first run the refutation, and then run verification if refutation does not produce a proof. The combined pipeline produces a proof (of refutation or of verification) for 128 of the 206 tasks, marking a recall of **57.3%** (118) and precision of **92.2%** (118 / 128), and with an overall F1 score of **0.71**.

Table 4 breaks the same run down by library tier. A few points worth noting:

- NFV struggles most with examples in the L3 tier for both the verification and refutation tasks, both in terms of precision and recall. The two have different causes. Recall is low because the imported libraries are hard to model, so the proof search more often gives up. The wrong verdicts are not caused by that modeling: Section 4.2.1 traces all four to preconditions the docstring never stated explicitly and to the argument types the translation chose.
- On the other hand, NFV does well in both L0 and L2b tiers, each with a single incorrect claim. L0 benefits from the reduced need to model libraries; L2b benefits from the fact that higher-order functions such as `filter` can be easily mapped to DAFNY’s comprehension support.
- Finally, within L2a, programs that directly manipulate strings prove at 4/10, compared with 11/15 for the remaining programs, suggesting that reasoning about Python string semantics in DAFNY remains challenging.

Table 5: Every wrong verdict of the reported run, by cause. *entry* is the kind of program the claim was made about: a *reference* is labeled correct and was refuted, a *mutant* is labeled incorrect and was proved. *unstated assumption* is the precondition the specification would have needed for the verdict and the label to agree. Every row is executed rather than asserted.

task	entry	cause	unstated assumption
HumanEval_11	reference	missing precondition	$ a = b $
HumanEval_20	reference	missing precondition	no NaN
HumanEval_49	reference	missing precondition	$p > 0$
HumanEval_64	reference	missing precondition	ASCII
HumanEval_79	reference	missing precondition	$decimal \geq 0$
HumanEval_111	reference	missing precondition	tokens are single letters
HumanEval_116	mutant	missing precondition	$arr[i] \geq 0$
HumanEval_6	mutant	stronger precondition	no empty space-delimited field
HumanEval_133	mutant	stronger precondition	1st is integer-typed
HumanEval_151	mutant	stronger precondition	1st is integer-typed
HumanEval_35	mutant	library formalization	—

4.2.1 Analysis

Eleven claims contradict a ground-truth label: 5 proofs of buggy programs and 6 counterexamples against correct ones. We investigated each of these and Table 5 attributes each to a cause.

4.2.2 Incorrect refutations

We assume that every reference implementation is correct with respect to the intent of its docstring, as we did when constructing the dataset in Section 2. A counterexample against a reference therefore leaves two possibilities: either the specification does not soundly formalize the docstring, or NFV mismodels the source code or its environment. Distinguishing the two can be subjective, because the intent of some docstrings is ambiguous. For example, the docstring for entry 49 (`modp`) documents “Return 2^n modulo p (be aware of numerics)”, which does not say whether p must be positive or n non-negative. We are conservative: we blame NFV whenever there is *any* interpretation of the docstring under which the specification is sound.

We arrive at the following conclusion: all 6 of them are real Python counterexamples to the postcondition as written, on inputs the docstring never excluded. For each, we name the missing precondition (underlined) and explain briefly how it blocks the buggy input.

- 11 (`string_xor`): $|a| = |b|$. The postcondition asserts $|rv| = |a| = |b|$, so on operands of unequal length it is unsatisfiable by *any* implementation.
- 20 (`find_closest_elements`): No NaN. The witness is a pair of equal elements with $\neg(x \leq x)$ — admitted because PyFloat is abstract, and realizable because every comparison with NaN is false.
- 49 (`modp`): $p > 0$. The witness is a negative modulus ($n = 0$, $p = -1$), which the theory’s requires `b != 0` admits and Python computes without complaint.
- 64 (`vowels_count`): ASCII-only s. Python lowercases the non-ASCII Latin capital I with dot above, `İ` (“`\u0130`”), to `i` followed by a combining dot, so the specification counts one vowel, whereas the implementation tests the original character against an ASCII vowel set and counts none.
- 79 (`decimal_to_binary`): decimal is nonnegative. For `-3`, Python returns “`-0b11`”; slicing from index 2 leaves “`b11`”, so the function returns “`dbb11db`”, which does not match `^db[01]+db$`.

- 111 (`histogram`): Tokens are single letters. The docstring “Given a string representing a space separated lowercase letters” seems to imply that a violating input such as “a aa” should not be considered.

4.2.3 Incorrect verifications

Spurious proofs come from a stronger precondition. 3 of the 5 are valid theorems about a program more constrained than the Python one.

- 6 (`parse_nested_parens`) has a *stronger precondition*. The docstring, in full, is “Input to this function is a string represented multiple groups for nested parentheses separated by spaces....” The inferred precondition includes a clause — *the string is nonempty, and no space is leading, trailing or doubled* — that rules out the buggy inputs ‘’, ‘()’, ‘()’ and ‘() ()’ on which the mutant fails.
- 133 (`sum_squares`) and 151 (`double_the_difference`): a *stronger precondition* manifested as an incorrect *type* signature: the parameter `lst` in Python is translated as `seq<int>` in DAFNY, which rejects the floating-point input on which the mutant fails.

Incorrect library formalism: 35 (`max_element`) computes the maximum of a list, and the specification asserts `return_value == max(1)`, where `max` is the Python built-in. The mutant adds 1 to the return value, violating that specification. The translated obligation is `result == PyMax(1)`, and `PyMax` is a function stage 1 wrote by transcribing the mutant’s own loop, its base case adding 1 to the result and so introducing the same bug into the specification.

Incorrect label: Finally, 116 (`sort_array`) is an entry where NFV changes the *label* of the mutant. The docstring for this example says “an array of non-negative integers” and the mutant fails the specification only for negative values. NFV installs a correct precondition and is therefore able to prove that the mutant satisfies the postcondition. The problem came not from the NL2POSTCOND dataset, but from the EVALPLUS dataset, which added a test case containing a negative integer that violates the precondition.

4.3 The bounded-verifier arm: CBMC

In this section, we discuss instantiating NFV with CBMC, a bounded program verifier, taking C as the formal language. Because CBMC does not support rich specification constructs, the agent writes C code for the libraries from their documentation. Given the bounded nature of the verification, we only focus on the effectiveness of this arm for refutation. Besides compiling without errors, the translated C program must be free of *undefined behavior (UB)*, which CBMC’s built-in UB checks establish: overflow, bounds, pointer and conversion checks.

CBMC issues 72 counterexamples, of which 65 refute a mutant program and 7 are false: that is a recall of **63%** over the 103 buggy programs at **90%** precision, against the DAFNY arm’s 46% at 89% on the same programs. This offers an alternative for refutation when bounded verification can scale. The precondition is again the dominant source of unsound refutations, the same diagnosis Section 4.2.1 reaches for the proof arm using DAFNY.

4.4 Weaker LLMs

Both arms were re-run with three further LLMs in place of `gpt-5.6-sol`, holding prompts, checks, backend and budget fixed.

Table 6: Comparison of different LLMs for the proof-based tasks. *proved* is over the 103 correct programs, *refuted* over the 103 buggy ones, *bad* counts claims contradicting the label, *prec* is precision over claims, and *F1* combines that precision with recall over the 103, counting an abstention as a wrong answer would count. *type* and *audit* split the entries that never reached the proof step: code that failed the type checker, and code that resolved but failed the artifact checks. Rows are ordered by decreasing verification *F1*. A row marked [†] was stopped before finishing: its refutation half is scored over the 203 entries that completed, not 206. The 3 that did not finish are the ones the model spent longest on.

	verification				refutation				no proof attempted	
	proved	bad	prec	F1	refuted	bad	prec	F1	type	audit
<code>gpt-5.6-sol</code>	72/103 (70%)	5	94%	0.80	47/103 (46%)	6	89%	0.60	1	9
<code>gpt-5-mini</code>	56/103 (54%)	38	60%	0.57	34/103 (33%)	17	67%	0.44	72	20
<code>claude-sonnet-4.6</code> [†]	41/103 (40%)	12	77%	0.53	39/101 (39%)	2	95%	0.55	0	159
<code>claude-haiku-4.5</code>	11/103 (11%)	2	85%	0.19	7/103 (7%)	0	100%	0.13	92	257

We observe that:

- `gpt-5.6-sol` is the only model that can provide value for this dataset, with the combined pipeline achieving **92%** precision and **57%** recall.
- `claude-haiku-4.5` performs quite poorly on both tasks. It also has the largest number of examples where the translated DAFNY code failed to type check or pass the audits.
- There is a sharp drop in the verification capability of weaker models compared to `gpt-5.6-sol` due to either reduced recall (`claude-sonnet-4.6`) or reduced precision (`gpt-5-mini`) or both. Both of these models have substantially more typechecking and audit failures.
- Degradation is task-asymmetric, and affects the verification tasks more than the refutation tasks.

4.5 Strawman and symbolic baselines

We also compare NFV against three baselines spanning a spectrum from fully neural to fully symbolic.

LLM-as-judge. In this baseline, the LLM is shown the program and the specification in Python — with the docstring’s input assumptions supplied — and asked whether the program satisfies the specification. It commits on every one of the 206 entries, never abstaining, and refutes 76% of a corpus. Overall, it is wrong on 58 of the 206 entries at 72% precision (compared to the 92% precision NFV provides through its ability to abstain when it cannot back its verdict with a proof artifact): it rejects 56 of the 103 correct programs and accepts 2 of the 103 incorrect ones. The failure is not only the error rate but what is left behind: an opinion, with no checkable artifact.

Unstaged agent-verifier. The second baseline asks for a verifiable DAFNY method carrying the specification, with a repair loop against the verifier with no restrictions — one call, no separated stages, nothing written blind, no freeze and no checks. It proves 98% of the correct programs and 98% of the known-wrong ones: precision 50%. However, the program and specification being proved are not the intended ones, since the agent is incentivized merely to find a proof: the code is rewritten (57% of unsound proofs), the goal is restated using invented library helpers (35%), or the proof rests on an explicit assumption (8%). These categories are mutually exclusive; additionally, 56% of the proofs contain an unjustified precondition, overlapping the primary categories. A proof is only as good as the artifact it is about; so NFV provides the discipline that fixes the artifact, not the proof search.

A sound verifier. We also explored NAGINI, a symbolic Python verifier [44] that leverages SMT solvers for proofs. Given a type-annotated program with contracts, it discharges obligations

comparable to the ones DAFNY discharges here. In its default configuration it proves **0%** on this corpus, because **83%** of the entries are rejected for the lack of type-annotated signatures, which these programs do not carry. Asking an agent to supply good guesses at these annotations raises the fraction that reach the verifier to 98%, but raises the proof rate only to **2%**. This result is not a judgment of the capability of NAGINI on this dataset, but rather a demonstration of the utility of NFV. We attribute the low success rate to the absence of a formal environment specification E_F (including preconditions that are vital even for tier L0 without any libraries) that symbolic verifiers assume, and to the absence of proof automation — precisely the burden that symbolic verification tools place on their end users.

5 Related work

Neuro-formal verification is inspired by decades of investment in symbolic program verification, its persistent challenges, and recent progress in AI agents. We describe a few (far from exhaustive) strands of prior work in these two domains.

5.1 Symbolic program verification

Translation-based verifiers: Verifying a program by translating it into an (intermediate) verification language is a long-standing practice. SPEC# [17] translates an annotated dialect of C#, VCC [14] and HAVOC [19] translate C, VERISOL [26] translates Solidity, and DAFNY [45] compiles its own source language, all of them into BOOGIE [16] and from there into SMT. CSLIB [18] is a proposal for developing Lean libraries for the semantics of languages and program logics, which will make these translations more trusted and reusable.

Extended-static checkers: NFV is inspired by works on *extended static checking* [33]: the verification is performed on a formal model of the source code, even though the formalism may be optimistic or “soundy” [32]. Such a translation either targets a fragment of the language or adopts optimistic assumptions that trade soundness for coverage of the code it can analyze. Tools (such as SDV, HAVOC, etc.) of this genre have been deployed on production software for “proofs” of properties in systems code against a formal model of code [34, 40, 41, 55], where the value delivered lies in hard-to-find defects that an engineer confirms and fixes.

Environment modeling: Most formal verification efforts work on closed programs and face the problem of modeling the environment of an *open program*, which leaves end-to-end soundness either in the hands of the user [27] or in heuristic inference of environment assumptions like angelic verification [30, 31]. Angelic verification and the earlier work on almost-correct specifications [29] were similarly motivated by the goal of a proof-based empirical separator of correct and incorrect programs, obtained purely through abductive inference [28] of environment assumptions.

We envision NFV helping to unlock the value of formal verification for mainstream developers without being constrained to specific languages and domains by leveraging progress in AI agents along three directions discussed below.

5.2 Agentic program reasoning

Neural language translation — the problem of translating across languages predates the current generation of agents and LLMs [20, 21, 22] but has advanced quickly with them [23]. NFV will benefit from defining datasets for translating from mainstream languages into verification-aware languages, where we ensure equivalence through tests or symbolic means.

Intent formalization — the process of generating and validating formal program specifications from informal documentation such as docstrings [49], a concept related to *autoformalization* [24] from formal mathematics. These specifications can be attested through symbolic validation against a program’s tests [50] without the need for implementations, to ensure their soundness, completeness and preciseness (modulo tests). Such work was previously performed manually, but has benefited from agentic proof automation [51]. This will alleviate the need for end-users of NFV to inspect non-trivial formal library specifications at the cost of inspecting concrete tests in the source language that are much more familiar to them.

Agentic proof automation — progress has been rapid in invariant synthesis with language models [54, 53], whole-proof generation and repair [43], retrieval-augmented tactic prediction [35], and proof generation for VERUS [36] and DAFNY [42]. We foresee continued benefits from improvements in frontier-model capability.

6 Conclusion and Future Work

We presented neuro-formal verification as a principled integration of AI coding agents and verification-aware languages to facilitate language-agnostic formal program reasoning. We envision NFV not only as a cost-effective way to find defects, but also as a means for developers to build confidence in their code gradually and to understand the assumptions required for it to behave correctly. We have demonstrated that recent progress in agentic reasoning is beginning to enable such formal program reasoning: the pipeline correctly resolves 57% of a dataset at 92% precision, backing every verdict it issues with a machine-checked proof. The dataset remains challenging for the current flagship model, which leaves the remaining 43% unresolved.

Much research needs to be done to bring this to mainstream software developers not versed in formal verification. We need to curate *labeled* datasets that are increasingly representative of real-world software development tasks covering richer libraries and concurrency. Increasing the trustworthiness of each agentic component through symbolic techniques will be helpful. For example, a benchmark of code translation into verification-aware languages, evaluated through tests or other symbolic means, can help make this component robust. Similarly, we need real-world benchmarks for intent formalization, and techniques to evaluate the quality of agent-generated specifications for the environment against existing artifacts such as tests [49, 50]. These advances should be grounded in realistic user studies to understand the utility of the approach, including the evaluation of informal concepts such as proof sketches. Finally, NFV needs to be integrated with modern software-development workflows. Lean Squad’s work [52] on generating Lean proofs from source code (without necessarily adopting NFV’s staged approach to isolating sources of unsoundness) can be viewed as an experiment in this space.

NFV does not replace proof with empirical evaluation: every verdict from the proof-based pipeline is backed by a machine-checked proof over formal models. Empirical evaluation instead measures how faithfully those models represent the source program, property, and environment. This end-to-end accuracy can be compared across foundation models and improved using data-driven methods, as for any learned component. This framing may offer a more holistic basis for bringing formal reasoning tools to mainstream developers.

Acknowledgments

The author is grateful to Thomas Ball for detailed feedback on an earlier preprint of this paper.

References

- [1] A. M. Turing. Checking a Large Routine. In *Report of a Conference on High Speed Automatic Calculating Machines*, Cambridge, 1949.
- [2] C. A. R. Hoare. An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12(10), 1969.
- [3] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.
- [4] E. Ioannidis, N. Swamy, G. Ebner, M. Philipose and T. Ramananandro. Proofs Promptly: Proof-Oriented Programming with AI Agents (Experience Report). *Proc. ACM Program. Lang.* 10(ICFP), Article 311, 2026.
- [5] C. Barrett, P. Fontaine and C. Tinelli. *The SMT-LIB Standard: Version 2.6*. Department of Computer Science, The University of Iowa, <https://smt-lib.org>, 2016.
- [6] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. In *TACAS*, 2008.
- [7] The Rocq Development Team. *The Rocq Prover* (formerly COQ). Inria, <https://rocq-prover.org>, 2025.
- [8] T. Nipkow, L. C. Paulson and M. Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer LNCS 2283, 2002.
- [9] N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss et al. Dependent Types and Multi-Monadic Effects in F*. In *POPL*, 2016.
- [10] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish et al. seL4: Formal Verification of an OS Kernel. In *SOSP*, 2009.
- [11] X. Leroy. Formal Verification of a Realistic Compiler. *Communications of the ACM*, 52(7), 2009.
- [12] K. Bhargavan, B. Bond, A. Delignat-Lavaud, C. Fournet, C. Hawblitzel, C. Hrițcu, S. Ishtiaq, M. Kohlweiss, R. Leino, J. Lorch et al. Everest: Towards a Verified, Drop-in Replacement of HTTPS. In *SNAPL*, 2017.
- [13] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat et al. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. In *OOPSLA*, 2024.
- [14] E. Cohen, M. Dahlweid, M. A. Hillebrand, D. Leinenbach, M. Moskal, T. Santen, W. Schulte and S. Tobies. VCC: A Practical System for Verifying Concurrent C. In *TPHOLs*, 2009.
- [15] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno and C. Hawblitzel. Verus: Verifying Rust Programs Using Linear Ghost Types. In *OOPSLA*, 2023.
- [16] M. Barnett, B.-Y. E. Chang, R. DeLine, B. Jacobs and K. R. M. Leino. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *FMCO*, LNCS 4111, 2006.

- [17] M. Barnett, K. R. M. Leino and W. Schulte. The Spec# Programming System: An Overview. In *CASSIS*, LNCS 3362, 2005.
- [18] C. W. Barrett, S. Chaudhuri, F. Montesi, J. Grundy, P. Kohli, L. de Moura, A. Rademaker and S. Yingchareonthawornchai. CSLib: The Lean Computer Science Library. *CoRR*, abs/2602.04846, 2026.
- [19] S. K. Lahiri and S. Qadeer. Back to the Future: Revisiting Precise Program Verification using SMT Solvers. In *POPL*, 2008.
- [20] B. Roziere, M.-A. Lachaux, L. Chanussot and G. Lample. Unsupervised Translation of Programming Languages. In *NeurIPS*, 2020.
- [21] B. Rozière, J. M. Zhang, F. Charton, M. Harman, G. Synnaeve and G. Lample. Leveraging Automated Unit Tests for Unsupervised Code Translation. In *ICLR*, 2022.
- [22] M. Szafraniec, B. Rozière, H. Leather, P. Labatut, F. Charton and G. Synnaeve. Code Translation with Compiler Representations. In *ICLR*, 2023.
- [23] Z. Yang, F. Liu, Z. Yu, J. W. Keung, J. Li, S. Liu, Y. Hong, X. Ma, Z. Jin and G. Li. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *Proc. ACM Softw. Eng.*, 1(FSE), 2024.
- [24] Y. Wu, A. Q. Jiang, W. Li, M. N. Rabe, C. Staats, M. Jamnik and C. Szegedy. Autoformalization with Large Language Models. In *NeurIPS*, 2022.
- [25] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sankar, L. P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha and R. Jabbarvand. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *ICSE*, 2024.
- [26] Y. Wang, S. K. Lahiri, S. Chen, R. Pan, I. Dillig, C. Born, I. Naseer and K. Ferles. Formal Verification of Workflow Policies for Smart Contracts in Azure Blockchain. In *VSTTE*, LNCS 12031, 2020.
- [27] S. Priya, T. Kahsai and A. Gurfinkel. Unlocking the Power of Environment Assumptions for Unit Proofs. In *SEFM*, LNCS 15280, 2024.
- [28] I. Dillig, T. Dillig and A. Aiken. Automated Error Diagnosis Using Abductive Inference. In *PLDI*, 2012.
- [29] S. Blackshear and S. K. Lahiri. Almost-Correct Specifications: A Modular Semantic Framework for Assigning Confidence to Warnings. In *PLDI*, 2013.
- [30] A. Das, S. K. Lahiri, A. Lal and Y. Li. Angelic Verification: Precise Verification Modulo Unknowns. In *CAV*, LNCS 9206, 2015.
- [31] S. K. Lahiri, A. Lal, S. Gopinath, A. Nutz, V. Levin, R. Kumar, N. Deisinger, J. Lichtenberg and C. Bansal. Angelic Checking within Static Driver Verifier: Towards high-precision defects without (modeling) cost. In *FMCAD*, 2020.
- [32] B. Livshits, M. Sridharan, Y. Smaragdakis, O. Lhoták, J. N. Amaral, B.-Y. E. Chang, S. Z. Guyer, U. P. Khedker, A. Møller and D. Vardoulakis. In Defense of Soundness: A Manifesto. *Communications of the ACM*, 58(2):44–46, 2015.

- [33] C. Flanagan, K. R. M. Leino, M. Lillibridge, G. Nelson, J. B. Saxe and R. Stata. Extended Static Checking for Java. In *PLDI*, 2002.
- [34] J. Vanegue and S. K. Lahiri. Towards Practical Reactive Security Audit Using Extended Static Checkers. In *IEEE Symposium on Security and Privacy*, 2013.
- [35] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger and A. Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *NeurIPS Datasets and Benchmarks*, 2023.
- [36] C. Yang, X. Li, M. R. H. Misu, J. Yao, W. Cui, Y. Gong, C. Hawblitzel, S. K. Lahiri, J. R. Lorch, S. Lu, F. Yang, Z. Zhou and S. Lu. AutoVerus: Automated Proof Generation for Rust Code. In *OOPSLA*, 2025.
- [37] A. Karpathy. Vibe coding. Post on X, February 2025.
- [38] F. Kirchner, N. Kosmatov, V. Prévosto, J. Signoles and B. Yakobowski. Frama-C: A Software Analysis Perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- [39] E. Clarke, D. Kroening and F. Lerda. A Tool for Checking ANSI-C Programs. In *TACAS*, 2004.
- [40] T. Ball, B. Hackett, S. K. Lahiri, S. Qadeer and J. Vanegue. Towards Scalable Modular Checking of User-Defined Properties. In *VSTTE*, LNCS 6217, 2010.
- [41] T. Ball, V. Levin and S. K. Rajamani. A Decade of Software Model Checking with SLAM. *Communications of the ACM*, 54(7):68–76, 2011.
- [42] M. R. H. Misu, C. V. Lopes, I. Ma and J. Noble. Towards AI-Assisted Synthesis of Verified Dafny Methods. In *FSE*, 2024.
- [43] E. First, M. N. Rabe, T. Ringer and Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *FSE*, 2023.
- [44] M. Eilers and P. Müller. Nagini: A Static Verifier for Python. In *CAV*, 2018.
- [45] K. R. M. Leino. Dafny: An Automatic Program Verifier for Functional Correctness. In *LPAR*, 2010.
- [46] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pondé de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
- [47] J. Liu, C. S. Xia, Y. Wang and L. Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [48] M. Endres, S. Fakhoury, S. Chakraborty and S. K. Lahiri. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? In *FSE*, 2024.
- [49] S. K. Lahiri. Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents. arXiv:2603.17150, 2026.

- [50] S. K. Lahiri. Evaluating LLM-driven User-Intent Formalization for Verification-Aware Languages. In *FMCAD*, 2024.
- [51] N. Swamy and S. K. Lahiri. Spotting Specification Gaps with Small Proof-Oriented Tests. *RiSE MSR Blog*, Microsoft Research, April 2026. <https://risemsr.github.io/blog/2026-04-16-spotting-specs/>.
- [52] D. Syme. Lean Squad: Automated Software Verification with Near-Zero Human Labour. Blog post, April 2026. <https://dsyme.net/2026/04/20/lean-squad-automated-software-verification-with-near-zero-human-labour/>.
- [53] H. Wu, C. Barrett and N. Narodytska. Lemur: Integrating Large Language Models in Automated Program Verification. In *ICLR*, 2024.
- [54] A. Kamath, A. Senthilnathan, S. Chakraborty, P. Deligiannis, S. K. Lahiri, A. Lal, A. Rastogi, S. Roy and R. Sharma. Finding Inductive Loop Invariants using Large Language Models. In *FMCAD*, 2024.
- [55] A. Lal, S. Qadeer, and S. K. Lahiri. A Solver for Reachability Modulo Theories. In *CAV*, 2012.
- [56] P. W. O’Hearn. Incorrectness Logic. *Proceedings of the ACM on Programming Languages*, 4(POPL), 2020.

A Dataset details

A.1 The benchmark in full

Table 7 lists every task in `nl2post-safe` with the postcondition its two entries are scored against, together with the upstream identifiers that pin the pair.

Table 7: The 103 pairs — all 206 entries — with the postcondition each pair is scored against, and the upstream identifiers that pin it. **task** is the HumanEval number. **p** is the `response_num` selecting the postcondition in `summary_eval_results.jsonl` and `raw_preprocessed_samples.jsonl`. **m** is the `response_num` selecting the mutant in `distinct_code_mutants.jsonl`; below 200 it is a natural mutant, at 200 and above an artificial one. The reference and the mutant of a task share a postcondition, so one row covers two entries. Postconditions are reproduced after sanitization (Appendix A), which is the form the pipeline sees.

task	entry point	p	m	postcondition
1	separate_paren_groups	2	0	assert all((s.count('(') == s.count(')') and '()' in s for s in return_value))
2	truncate_number	0	202	assert 0 <= return_value < 1
3	below_zero	0	200	assert return_value == True and any((sum(operations[:i + 1]) < 0 for i in range(len(operations)))) or return_value == False
4	mean_absolute_deviation	1	213	assert 0 <= return_value
5	intersperse	0	27	assert len(return_value) == len(numbers) * 2 - 1 if numbers else len(return_value) == 0
6	parse_nested_parens	1	7	assert len(return_value) == len(paren_string.split())
8	sum_product	0	49	assert sum(numbers) == return_value[0]
9	rolling_max	0	106	assert all((return_value[i] >= max(numbers[:i + 1]) for i in range(len(numbers))))

task	entry point	p	m	postcondition
10	make_palindrome	0	0	assert return_value.startswith(string) and return_value == return_value[::-1]
11	string_xor	0	6	assert len(return_value) == len(a) == len(b) and all((ch in '01' for ch in return_value))
13	greatest_common_divisor	0	205	assert a % return_value == 0 and b % return_value == 0
17	parse_music	2	202	assert len(return_value) == len([note for note in music_string.split() if note in ['o', 'o', '. ']])
19	sort_numbers	0	3	assert return_value == ' '.join(sorted(numbers.split(), key=lambda x: ['zero', 'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine'].index(x)))
20	find_closest_elements	1	147	assert return_value in [(x, y) for i, x in enumerate(numbers) for j, y in enumerate(numbers) if i != j and x <= y]
21	rescale_to_unit	1	200	assert len(numbers) == len(return_value)
25	factorize	1	202	assert n == eval('*'.join(map(str, return_value)))
26	remove_duplicates	0	205	assert len(return_value) == len(set(return_value))
30	get_positive	0	200	assert all((i > 0 for i in return_value))
31	is_prime	0	27	assert n < 2 and (not return_value) or n >= 2
33	sort_third	0	5	assert all((l[i] == return_value[i] for i in range(len(l)) if i % 3 != 0))
34	unique	0	200	assert return_value == sorted(list(set(l)))
35	max_element	0	201	assert return_value == max(l)
36	fizz_buzz	0	205	assert isinstance(return_value, int) and return_value >= 0
37	sort_even	5	11	assert sorted(return_value[::2]) == sorted(l[::2])
41	car_race_collision	0	7	assert isinstance(return_value, int) and return_value >= 0
42	incr_list	0	200	assert all((return_value[i] == l[i] + 1 for i in range(len(l))))
43	pairs_sum_to_zero	5	42	assert not return_value or any((x + y == 0 for i, x in enumerate(l) for j, y in enumerate(l) if i != j))
44	change_base	0	60	assert int(return_value, base) == x
45	triangle_area	0	200	assert return_value == 0.5 * a * h
47	median	4	200	assert sum((i <= return_value for i in l)) >= len(l) // 2 and sum((i >= return_value for i in l)) >= len(l) // 2
49	modp	1	1	assert 0 <= return_value < p
50	decode_shift	9	289	assert len(s) == len(return_value)
51	remove_vowels	0	200	assert all((c not in 'aeiouAEIOU' for c in return_value))
53	add	0	200	assert return_value == x + y
56	correct_bracketing	0	200	assert return_value == False or brackets.count('<') == brackets.count('>')
58	common	0	203	assert return_value == sorted(return_value)
59	largest_prime_factor	1	44	assert n % return_value == 0
60	sum_to_n	0	189	assert return_value == n * (n + 1) // 2
61	correct_bracketing	0	201	assert return_value == False or brackets.count('(') == brackets.count('')
64	vowels_count	0	0	assert return_value == s.lower().count('a') + s.lower().count('e') + s.lower().count('i') + s.lower().count('o') + s.lower().count('u') + (1 if s.lower().endswith('y') else 0)
65	circular_shift	0	7	assert len(str(x)) == len(return_value)

task	entry point	p	m	postcondition
66	digitSum	6	200	assert sum((ord(char) for char in s if char.isupper())) == return_value
67	fruit_distribution	0	15	import re assert isinstance(return_value, int) and return_value >= 0 and (return_value == n - sum(map(int, re.findall('\\d+', s))))
68	pluck	0	62	assert isinstance(return_value, list) and (len(return_value) == 2 and return_value[0] % 2 == 0 and (arr[return_value[1]] == return_value[0])) or (return_value == [] and all((i % 2 != 0 for i in arr)))
69	search	1	1	assert return_value == max([num for num in set(lst) if lst.count(num) >= num], default=-1)
70	strange_sort_list	0	48	assert len(lst) == len(return_value)
71	triangle_area	0	210	assert isinstance(return_value, float) and round(return_value, 2) == return_value or return_value == -1
73	smallest_change	2	0	assert 0 <= return_value <= len(arr) // 2
77	iscube	2	0	assert return_value == (round(abs(a) ** (1.0 / 3)) ** 3 == abs(a))
78	hex_key	1	203	assert isinstance(return_value, int) and 0 <= return_value <= len(num)
79	decimal_to_binary	0	6	import re assert re.match('^db[01]+db\$', return_value)
81	numerical_letter_grade	0	215	assert len(grades) == len(return_value)
86	anti_shuffle	2	0	assert len(s) == len(return_value)
87	get_row	0	3	assert all((return_value[i][0] < return_value[i + 1][0] or (return_value[i][0] == return_value[i + 1][0] and return_value[i][1] >= return_value[i + 1][1]) for i in range(len(return_value) - 1)))
88	sort_array	0	206	assert len(array) == len(return_value)
89	encrypt	0	3	assert len(s) == len(return_value)
90	next_smallest	3	6	assert (return_value is None) == (len(set(lst)) < 2)
93	encode	1	17	assert len(return_value) == len(message)
96	count_up_to	3	1	assert all((i < n and all((i % j != 0 for j in range(2, int(i ** 0.5) + 1))) for i in return_value))
97	multiply	0	0	assert return_value == abs(a) % 10 * (abs(b) % 10)
99	closest_integer	9	9	assert isinstance(return_value, int) and abs(return_value - float(value)) <= 1
100	make_a_pile	1	0	assert len(return_value) == n and all((return_value[i] == return_value[i - 1] + 2 for i in range(1, n)))
102	choose_num	0	5	assert return_value == -1 or (x <= return_value <= y and return_value % 2 == 0 and all((i <= return_value for i in range(x, y + 1) if i % 2 == 0)))
103	rounded_avg	0	12	assert isinstance(return_value, str) and return_value.startswith('0b') and all((c in '01' for c in return_value[2:])) or return_value == -1
104	unique_digits	0	126	assert all((str(i)[j] in '13579' for i in return_value for j in range(len(str(i))))) and return_value == sorted(return_value)
105	by_length	1	203	assert all((item in ['One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine'] for item in return_value)) and len(return_value) <= len(arr)
106	f	0	15	assert len(return_value) == n

task	entry point	p	m	postcondition
107	even_odd_palindrome	0	140	assert sum(return_value) == len([i for i in range(1, n + 1) if str(i) == str(i)[::-1]])
108	count_nums	2	59	assert isinstance(return_value, int) and 0 <= return_value <= len(arr)
110	exchange	4	0	assert (return_value == 'YES') == (len([num for num in lst1 if num % 2 != 0]) <= len([num for num in lst2 if num % 2 == 0]))
111	histogram	7	18	assert all([isinstance(key, str) and key.islower() and isinstance(value, int) and (value == test.count(key)) for key, value in return_value.items()])
112	reverse_delete	0	205	assert return_value[1] == (return_value[0] == return_value[0][::-1])
113	odd_count	0	220	assert len(lst) == len(return_value)
114	minSubArraySum	4	0	assert return_value == min((sum(nums[i:j]) for i in range(len(nums)) for j in range(i + 1, len(nums) + 1)))
115	max_fill	0	0	assert isinstance(return_value, int) and return_value >= 0
116	sort_array	8	9	assert all((bin(return_value[i]).count('1') <= bin(return_value[i + 1]).count('1') and (bin(return_value[i]).count('1') != bin(return_value[i + 1]).count('1') or return_value[i] <= return_value[i + 1]) for i in range(len(return_value) - 1)))
117	select_words	7	4	assert all((sum((1 for c in word if c.lower() not in 'aeiou ')) == n for word in return_value)) and all((word in return_value for word in s.split() if sum((1 for c in word if c.lower() not in 'aeiou ')) == n))
118	get_closest_vowel	7	11	assert return_value == '' or return_value in 'AEIOUaeiou'
120	maximum	0	0	assert len(return_value) == k and all((return_value[i] <= return_value[i + 1] for i in range(len(return_value) - 1)))
121	solution	1	0	assert isinstance(return_value, int) and return_value == sum((x for i, x in enumerate(lst) if i % 2 == 0 and x % 2 == 1))
123	get_odd_collatz	0	43	assert return_value == sorted(return_value) and all((num % 2 != 0 for num in return_value))
127	intersection	0	235	assert return_value in ['YES', 'NO']
128	prod_signs	2	77	assert isinstance(return_value, int) and len(arr) > 0 or (return_value is None and len(arr) == 0)
130	tri	0	3	assert len(return_value) == n + 1
131	digits	0	18	assert return_value == 0 if all((int(digit) % 2 == 0 for digit in str(n))) else True
133	sum_squares	0	151	assert isinstance(return_value, int) and return_value >= 0
135	can_arrange	0	0	assert return_value == -1 and sorted(arr) == arr or (return_value > 0 and arr[return_value] < arr[return_value - 1])
136	largest_smallest_integers	0	115	assert return_value[0] == max(filter(lambda x: x < 0, lst), default=None) and return_value[1] == min(filter(lambda x: x > 0, lst), default=None)
141	file_name_check	0	42	assert return_value == 'Yes' and (file_name[0].isalpha() and file_name.split('.')[1] in ['txt', 'exe', 'dll']) or return_value == 'No'

task	entry point	p	m	postcondition
143	words_in_sentence	1	200	assert all((word in sentence.split() and len(word) in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97] for word in return_value.split()))
145	order_by_points	3	134	assert len(nums) == len(return_value)
146	specialFilter	0	49	assert return_value == len([num for num in nums if num > 10 and int(str(num)[0]) % 2 != 0 and (int(str(num)[-1]) % 2 != 0)])
148	bf	1	205	assert isinstance(return_value, tuple) and all((isinstance(planet, str) for planet in return_value))
150	x_or_y	0	113	assert return_value in {x, y}
151	double_the_difference	1	4	assert isinstance(return_value, int) and return_value >= 0
152	compare	1	206	assert len(return_value) == len(game) == len(guess)
153	Strongest_Extension	1	4	assert return_value == class_name + '.' + max(extensions, key=lambda x: sum((1 for c in x if c.isupper())) - sum((1 for c in x if c.islower())))
155	even_odd_count	0	8	assert sum(return_value) == len(str(abs(num)))
156	int_to_mini_roman	1	203	assert isinstance(return_value, str) and all((char in 'ivxlc dm' for char in return_value))
157	right_angle_triangle	1	16	assert return_value == True and (a ** 2 + b ** 2 == c ** 2 or a ** 2 + c ** 2 == b ** 2 or b ** 2 + c ** 2 == a ** 2) or (return_value == False and (not (a ** 2 + b ** 2 == c ** 2 or a ** 2 + c ** 2 == b ** 2 or b ** 2 + c ** 2 == a ** 2)))
159	eat	2	4	assert return_value == [min(number + need, number + remaining), max(0, remaining - need)]
161	solve	0	1	assert all((ch.isalpha() and ch.swapcase() == s[i] or (not ch.isalpha() and (ch == s[i] or ch == s[-(i + 1)])) for i, ch in enumerate(return_value)))
163	generate_integers	0	128	assert all((i % 2 == 0 for i in return_value))

B An example program from each library tier

L0 — HumanEval/157. It uses nothing beyond arithmetic, indexing and control flow. Nothing outside the program has to be described, so the model’s trusted base can be empty.

```
def right_angle_triangle(a, b, c):
    return a ** 2 + b ** 2 == c ** 2 or a ** 2 + c ** 2 == b ** 2 or b ** 2 + c **
        2 == a ** 2
```

L1 — HumanEval/121. Beyond arithmetic, indexing and control flow it uses `sum`. Each call here is one self-contained, total function whose meaning the model can state in a line.

```
def solution(lst):
    return sum((lst[i] for i in range(len(lst)) if i % 2 == 0 and lst[i] % 2 == 1)
        )
```

L2a — HumanEval/120. Beyond arithmetic, indexing and control flow it uses `.sort`, `sorted`. A faithful specification has to relate whole collections — what the result contains, and in what order — rather than give a single value.

```
def maximum(arr, k):  
    return sorted(sorted(arr)[::-1][:k])
```

L2b — HumanEval/30. Beyond arithmetic, indexing and control flow it uses `filter`, `list`. The specification must quantify over an argument whose body it cannot see, which is a different demand from describing a collection.

```
def get_positive(l: list):  
    return list(filter(lambda x: x > 0, l))
```

L3 — HumanEval/133. Beyond arithmetic, indexing and control flow it uses `import math`, `map`, `math.ceil`, `sum`. The boundary is not the model's to choose: it must axiomatise a surface defined elsewhere, and any axiom it writes about that surface is assumed, not checked.

```
def sum_squares(lst):  
    import math  
    return sum(map(lambda x: math.ceil(x) ** 2, lst))
```